Entrega 7 — Data Integration: Minha Rota

A. Sistemas Integrados

Para ilustrar a integração na arquitetura do Minha Rota, destacamos dois fluxos principais de troca de informações:

Fluxo 1: Operação em Tempo Real (App e Backend)

- **Qual sistema envia os dados?** O App Móvel (utilizado pelo Motorista).
- **Qual sistema recebe os dados?** A API Backend (Minha Rota API).
- **Por que os sistemas precisam trocar informações?** O motorista precisa atualizar o sistema em tempo real quando um colaborador entra no autocarro (check-in), para que o RH acompanhe o embarque e a plataforma registe a viagem para efeitos de faturamento e auditoria.

Fluxo 2: Análise de Dados (Banco Operacional e DW)

- **Qual sistema envia os dados?** O Banco Operacional (PostgreSQL).
- **Qual sistema recebe os dados?** O Data Warehouse (Google BigQuery).
- **Por que os sistemas precisam trocar informações?** Para retirar a carga de relatórios pesados do banco do dia a dia. Os dados operacionais precisam ser consolidados e enviados para o DW para permitir a criação de dashboards de BI para a diretoria.

B. Dados Integrados

Focando no **Fluxo 1 (Check-in do Passageiro)**, que representa a transação mais crítica do negócio:

- **Quais dados serão enviados?** A confirmação de que um colaborador embarcou ou faltou.
- **Quais dados serão recebidos (Resposta da API)?** Uma mensagem de sucesso e a atualização do número de vagas disponíveis no veículo.
- **Qual é o formato dos dados?** JSON (JavaScript Object Notation).
- **Quais campos são necessários?** ID da viagem, ID do colaborador, status do embarque e o timestamp (data/hora exata do embarque).

- **Existem transformações nos dados durante a integração?** Sim. O backend recebe o JSON simples do aplicativo, valida se o token JWT pertence àquele motorista, cruza o id_colaborador com o banco de dados para buscar o nome da empresa correspondente e transforma essa ação num comando INSERT/UPDATE no PostgreSQL, além de criar uma cópia da ação (log) enviando-a para o MongoDB.

Exemplo do Payload JSON (Motorista enviando o Check-in):

```
{
  "id_viagem": 450,
  "id_colaborador": 1024,
  "status_embarque": "EMBARCOU",
  "data_hora_leitura": "2026-11-03T07:15:30Z"
}
```

Exemplo da Resposta JSON (Backend respondendo ao App):

```
{
  "status": "sucesso",
  "mensagem": "Embarque registrado com sucesso",
  "vagas_restantes": 12
}
```

C. Forma de Integração

1. Para a comunicação entre App Móvel/Painel Web e Backend:

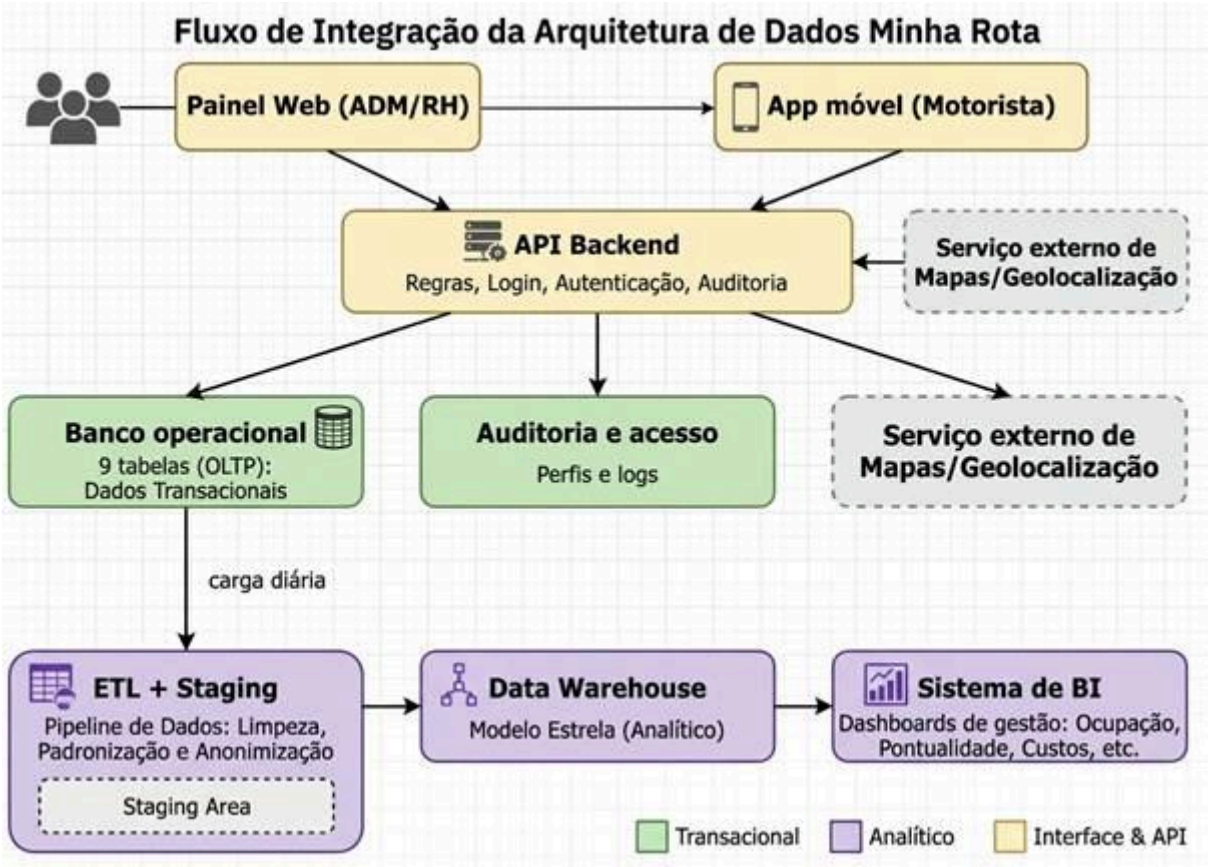
- **Mecanismo utilizado:** REST API (sobre protocolo HTTPS).
- **Por que é adequado?** O modelo REST é o padrão da indústria para aplicativos móveis. Ele é leve (usa JSON), *stateless* (não sobrecarrega o servidor com sessões ativas, ideal para áreas onde o motorista pode perder o sinal de internet por breves momentos) e permite respostas rápidas em tempo real. Além disso, integra perfeitamente com os frameworks modernos (como o React Native usado no aplicativo móvel).

2. Para a comunicação entre Banco Operacional e Data Warehouse:

- **Mecanismo utilizado:** Banco de dados via pipeline de ETL (Extract, Transform, Load) / Arquivo em lote.
- **Por que é adequado?** O ETL fará consultas (queries) programadas de madrugada no banco operacional, extraindo as viagens do dia concluídas, transformando-as (limpando dados sensíveis da LGPD) e carregando-as em massa no DW. Usar chamadas REST para mover milhões de linhas de análise histórica seria lento e ineficiente; a integração direta entre bases de dados é a solução robusta para *Big Data*.

D. Fluxo de Dados (Integração)

O fluxo de integração do ecossistema Minha Rota conecta o ambiente transacional interno com serviços externos e a camada analítica.



Integração Operacional (API & Webhooks): A API Backend atua como orquestradora central, comunicando-se com o Banco Operacional (OLTP) e validando rotas junto ao Serviço de Mapas/Geolocalização externo.

Integração Analítica (ETL/ELT): Extração diária de dados alterados no OLTP para a Staging Area, onde passam por saneamento e anonimização de dados pessoais antes de abastecer o Data Warehouse (Modelo Estrela).

E. Tratamento de Erros

Para garantir a resiliência e confiabilidade do sistema Minha Rota, as falhas nas integrações e transações são tratadas da seguinte forma:

- **Sistema de destino indisponível:** Utilização de políticas de Retry (tentativas automáticas com espaçamento exponencial) e o padrão Circuit Breaker na API. Caso o serviço externo (ex: Mapas) caia, a API utiliza dados em cache para operações críticas ou enfileira a requisição para processamento posterior.
- **Dados inválidos:** Validação estrita na borda (DTOs e JSON Schema na API Backend). Se o payload contiver campos malformados ou regras de negócio violadas, a requisição é rejeitada imediatamente com código HTTP adequado (ex: 400 Bad Request) e registrada na auditoria.
- **Timeout:** Definição de limites estritos de tempo de espera (timeouts) nas requisições de rede. Se o tempo limite for excedido, a transação é cancelada para evitar travamentos de threads, disparando um alerta para a equipe técnica.
- **Dados duplicados:** Implementação de chaves de idempotência (identificadores únicos por requisição) e restrições de unicidade (UNIQUE constraints) no Banco Operacional (ex: CPF, matrícula do colaborador), garantindo que envios repetidos acidentalmente não gerem registros duplicados.
- **Falha durante a transmissão:** Uso de protocolos seguros (HTTPS) com confirmação de recebimento por pacotes (ACK) e assinaturas de integridade. Em caso de queda de conexão a meio da transmissão, o cliente reencancia a requisição a partir do último estado confirmado.
- **Falha ao salvar os dados:** Uso de transações atômicas de banco de dados (ACID / blocos BEGIN ... COMMIT / ROLLBACK). Se ocorrer qualquer erro ao gravar no banco operacional, a transação inteira é desfeita, mantendo a consistência e evitando estados corrompidos ou parciais.

F. Consistência dos Dados

- **Como garantir que os dados recebidos estejam corretos?** Através de validações rigorosas de tipo, formato e integridade referencial na entrada da API Backend, combinadas com restrições de domínio diretamente no esquema do Banco Operacional (chaves estrangeiras e regras de validação para turnos, matrículas e CPFs).
- **Como evitar duplicidade?** Mediante o uso de chaves primárias e restrições de unicidade (UNIQUE) nas colunas de negócio do banco de dados (como CPF, e-mail e placa de veículos), além de validações lógicas na camada de serviço antes de efetivar o cadastro.
- **Como identificar uma atualização?** Utilizando colunas de controle temporal e de versão, como data_ultima_modificacao (timestamp atualizado automaticamente a cada UPDATE) e IDs imutáveis, permitindo que o processo de ETL identifique exatamente o que mudou desde a última extração.
- **Como tratar dados inconsistentes entre os sistemas?** Por meio de rotinas de reconciliação diária (reconciliation jobs) executadas no pipeline de ETL, geração de relatórios de divergência para os administradores corrigirem cadastros corrompidos no Painel Web, e validações cruzadas com o Banco de Auditoria.